



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2333 — Sistemas Operativos y Redes — 2/2026

Tarea 0

Composición: Tarea en parejas

Fecha de entrega: Miércoles 2 de septiembre a las 22:00 horas

● Objetivos

- crear y administrar procesos con `fork`, `exec` y `waitpid`;
- comunicar procesos mediante `pipe`;
- utilizar señales para controlar procesos hijos;
- mantener el estado de varios procesos concurrentes sin bloquear la interacción con el usuario;
- cerrar correctamente memoria y descriptores de archivo utilizados por el programa.

fazwatch



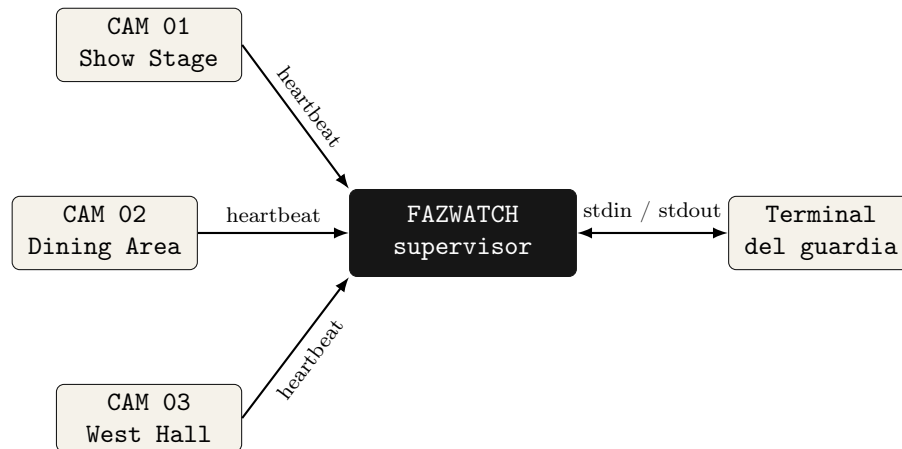
Son las 12:00 AM en Freddy Fazbear's Pizza. El local está cerrado, pero el sistema de cámaras continúa encendido. Cada zona del edificio es vigilada por una cámara controlada por un programa independiente. Algunas cámaras funcionan toda la noche; otras pueden dejar de responder, cerrarse inesperadamente o negarse a apagarse cuando el sistema se los solicita.

Tu misión es implementar **fazwatch**, la terminal central del guardia nocturno. Desde ella deberán iniciar cámaras, consultar su estado, pausarlas, reanudarlas y cerrarlas. Al mismo tiempo, **fazwatch** deberá detectar automáticamente cuándo una cámara ha dejado de reportar actividad.

REC • SECURITY LOG // 12:00 AM

// // // // //

● 1. La noche comienza



Cada cámara será un **proceso hijo independiente** y cada una tendrá un canal privado para enviar mensajes a **fazwatch**. El supervisor deberá seguir atendiendo al resto de las cámaras y a la terminal incluso si una cámara deja de responder.

● 2. Conceptos básicos

2.1 Heartbeat

Un *heartbeat* es un mensaje pequeño que un proceso envía periódicamente para indicar que sigue funcionando. En esta tarea el mensaje será simplemente el carácter:

```
H
```

Si una cámara lleva demasiado tiempo sin enviar un heartbeat, **fazwatch** asumirá que dejó de responder. El supervisor no debe asumir que todas las cámaras envían heartbeats con la misma frecuencia, este debe utilizar el tiempo transcurrido desde el último heartbeat recibido por una cámara para detectar una falla.

2.2 File descriptors

En Linux, un proceso accede a archivos, terminales, pipes y otros recursos mediante números enteros llamados *file descriptors* (FD). Al iniciar un programa normalmente ya existen los siguientes:

FD	Nombre	Uso
0	stdin	entrada estándar
1	stdout	salida estándar
2	stderr	salida de errores

Las syscalls **read** y **write** trabajan directamente con estos descriptores, por ejemplo: **write(1, ...)** escribe en la salida estándar.

Un file descriptor no es necesariamente un archivo. También puede representar un extremo de un pipe, que es precisamente lo que utilizaremos para comunicar las cámaras con **fazwatch**.

2.3 Pipes

Un **pipe** es un canal unidireccional mantenido por el sistema operativo. Se crea mediante:

```
int pipefd[2];
pipe(pipefd);
```

Si la llamada es exitosa:

```
pipefd[0]  extremo de lectura,
pipefd[1]  extremo de escritura.
```

Los descriptors abiertos se heredan al hacer **fork**. Por lo tanto, es posible crear un pipe antes de hacer un **fork**, dejar el extremo de escritura en el hijo y el extremo de lectura en el padre.

El siguiente ejemplo muestra esta idea:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int pipefd[2];
    pipe(pipefd);

    pid_t pid = fork();

    if (pid == 0) {
        /* Hijo solo escribe. */
        close(pipefd[0]);

        char message = 'H';
        write(pipefd[1], &message, 1);

        close(pipefd[1]);
        exit(0);
    }

    /* Padre solo lee. */
    close(pipefd[1]);

    char message;
    read(pipefd[0], &message, 1);
    printf("Recibi: %c\n", message);

    close(pipefd[0]);
    waitpid(pid, NULL, 0);
}
```

Documentación recomendada:

- <https://man7.org/linux/man-pages/man2/pipe.2.html>

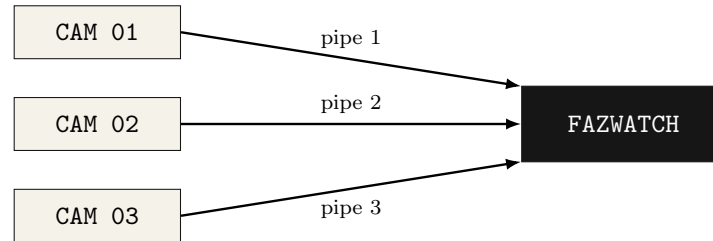
- <https://man7.org/linux/man-pages/man7/pipe.7.html>

REC • SECURITY LOG // 2:00 AM

// // // // //

● 3. Comunicación de cámaras

Cada cámara tendrá un **pipe exclusivo** hacia **fazwatch**. Las cámaras solamente escribirán y el supervisor solamente leerá.



Para esta tarea les entregaremos el código de algunas cámaras las cuales siempre escribirán sus mensajes hacia el file descriptor número 3. Por ejemplo:

```
#define FAZWATCH_FD 3

char heartbeat = 'H';
write(FAZWATCH_FD, &heartbeat, 1);
```

Por lo tanto, al iniciar una cámara, **fazwatch** deberá preparar un pipe, asegurándose que el extremo de escritura pueda ser usado por el nuevo programa hijo mediante el FD 3.

Hint: Un file descriptor sigue funcionando después de usar **exec**. Puede ser útil investigar **dup2** para controlar qué número de descriptor utilizará el programa ejecutado.

<https://man7.org/linux/man-pages/man2/dup.2.html>

3.1 Protocolo de heartbeat

El protocolo entre una cámara y **fazwatch** es el siguiente:

- El byte 'H' representa un heartbeat
- Cada cámara puede enviar heartbeats con una frecuencia distinta
- Una cámara puede terminar normalmente, terminar por una señal o dejar de enviar heartbeats sin finalizar
- Algunas cámaras pueden ignorar **SIGTERM**

● 4. Diseño de la terminal

Recapitulando, **fazwatch** es un sistema de monitoreo cuyo objetivo es manejar distintas camaras y verificar que cumplan el protocolo de heartbeat antes explicado. Además, **fazwatch** deberá exponer una terminal de comandos para que un usuario pueda interactura directamente con ella.

4.1 Estados

Cada cámara siempre debe encontrarse en uno de los siguientes estados:

Estado	Significado
RUNNING	La cámara está ejecutándose
PAUSED	Fue pausada por el usuario
TERMINATING	Se envió SIGTERM
FINISHED	El proceso terminó y fue recolectado

4.2 Ejecución

El programa debe compilarse mediante **make** y generar un ejecutable llamado **fazwatch**. Su ejecución será:

```
./fazwatch <timeout>
```

timeout es la cantidad máxima de segundos que una cámara en estado **RUNNING** puede permanecer sin enviar un heartbeat. Se puede asumir que será un entero positivo.

Podrán existir como máximo 10 cámaras activas simultáneamente. La terminal deberá aceptar los comandos que se explican a continuación.

4.3 Comandos

4.3.1 launch <executable>[arg1 ... argN]

Este comando sirve para iniciar una nueva cámara. **fazwatch** deberá crear un proceso hijo, crear un pipe para asignarle y luego ejecutar el programa solicitado. El nuevo programa debe disponer del extremo de escritura de su pipe mediante el FD 3 mientras que el supervisor debe conservar el extremo de lectura.

Ejemplo:

```
fazwatch> launch ./camera_normal 15
Camara iniciada con PID 4218.
```

4.3.2 status

Muestra todas las cámaras lanzadas durante la ejecución actual. Como mínimo debe indicar:

- PID

- Nombre o ruta del ejecutable
- Estado actual
- Tiempo transcurrido desde el último heartbeat válido
- Exit code, o -1 si no corresponde
- Señal de término, o -1 si no corresponde.

4.3.3 pause <pid>

Pausa una cámara en estado **RUNNING** mediante **SIGSTOP**. Mientras una cámara esté pausada, su timeout de heartbeat no debe avanzar.

4.3.4 resume <pid>

Reanuda una cámara **PAUSED** mediante **SIGCONT**. Al reanudarla, su tiempo hasta el siguiente timeout debe reiniciarse. Por ejemplo, si una cámara le faltaba un segundo para entrar en timeout cuando fue pausada, al momento de reanudar su ejecución, al cámara volverá a tener **timeout** segundos para mandar un heartbeat.

4.4.5 terminate <pid>

Solicita finalizar una cámara mediante **SIGTERM**. La cámara pasa a estado **TERMINATING**. Si después de 2 segundos continúa activa, se debe mandar **SIGKILL**.

Solo se puede intentar terminar camaras que esten en el estado **RUNNING**

4.4.6 shutdown

Inicia el apagado de **fazwatch**. No se aceptarán nuevos comandos. Todas las cámaras que no hayan finalizado deben recibir **SIGTERM**; después de 2 segundos, las que continúen activas deben recibir **SIGKILL**. El programa termina únicamente después de recolectar a todos sus hijos y cerrar sus descriptores.

● 5. Supervisión de cámaras

Cada vez que una cámara **RUNNING** envía un heartbeat válido, se actualiza su instante de última actividad. Si transcurren al menos **timeout** segundos sin recibir otro heartbeat, **fazwatch** debe:

1. informar que la cámara dejó de responder, por ejemplo, con un mensaje en la terminal.
2. enviar **SIGTERM**
3. cambiarla a **TERMINATING**
4. comenzar un período de gracia de 2 segundos
5. enviar **SIGKILL** si al terminar ese período la cámara sigue activa.

No bloquear la central

Mientras supervisa las cámaras, **fazwatch** debe continuar recibiendo y procesando comandos desde su terminal. Ninguna operación asociada a una cámara puede bloquear la ejecución de la terminal durante un período prolongado.

Esto significa que no es válido quedar detenido esperando a que una cámara escriba en su pipe, esperar de forma bloqueante a que termine un proceso hijo, ni detener la ejecución mientras transcurre el período de gracia posterior al envío de **SIGTERM**. Durante estos períodos, **fazwatch** debe continuar atendiendo eventos provenientes de las demás cámaras y comandos ingresados por el usuario.

No se permite utilizar threads para resolver este requerimiento ni utilizar **sleep** como mecanismo principal de espera.

Hint: Se recomienda investigar mecanismos que permitan observar varios file descriptors y saber cuáles tienen información disponible. Una alternativa disponible es **poll**.

<https://man7.org/linux/man-pages/man2/poll.2.html>

● 6. Procesos y descriptores

Recolección de hijos

fazwatch no debe dejar procesos zombie ni bloquearse esperando que una cámara termine. Se espera el uso **waitpid** dentro del ciclo principal, pudiendo recolectar más de un hijo terminado.

Al recolectar una cámara se debe guardar su exit code si terminó normalmente, o la señal correspondiente si terminó debido a una señal. Luego pasa a **FINISHED**.

Cierre de file descriptors

Después de un **fork**, el hijo hereda los descriptores que estaban abiertos en el padre. Cada proceso debe cerrar los extremos y pipes que no necesita. En particular:

- el supervisor debe cerrar los extremos de escritura de los pipes de las cámaras
- una cámara no debe mantener abiertos pipes pertenecientes a otras cámaras
- al finalizar una cámara, el supervisor debe cerrar su descriptor de heartbeat cuando corresponda.

Un programa puede parecer funcionar con una sola cámara y fallar al lanzar varias si mantiene descriptores abiertos innecesariamente. El manejo correcto de estos recursos forma parte de la evaluación.

● 7. Material entregado

Junto al enunciado se entregará:

- **protocol.h**: constantes del protocolo, incluido el FD reservado y el byte de heartbeat
- **camera_example.c**: código fuente de una cámara mínima de ejemplo
- **camera_normal**: cámara que reporta actividad y termina normalmente
- **camera_short**: cámara que termina por sí sola después de algunos heartbeats

- `camera_silent`: cámara que deja de enviar heartbeats sin terminar
- `camera_stubborn`: cámara que no termina al recibir `SIGTERM`

Durante la corrección podrán utilizarse cámaras adicionales que sigan el protocolo. Por ello, tu solución debe ser genérica y no depender únicamente de los ejecutables entregados. Se recomienda crear tus propias cámaras para probar que tu programa funcione en distintas situaciones.

● 8. Supuestos

- `timeout` siempre será un entero positivo
- Existirán como máximo 10 cámaras activas simultáneamente
- Los comandos tendrán una longitud máxima de 128 caracteres
- Las cámaras enviarán únicamente mensajes válidos del protocolo
- Los procesos hijos no crearán otros procesos
- Al momento de corregir, nunca se usará un `timeout` igual al ciclo de heartbeat de una cámara. Por lo tanto, no es necesario manejar el caso en el que, en un mismo segundo, una cámara cumple su `timeout` y a la vez manda un heartbeat.

REC • 6:00 AM // SHIFT COMPLETE

// // // // //

● 10. Formato de Entrega

A cada alumno se le asignó un nombre de usuario y una contraseña para el servidor del curso¹. Para entregar su tarea usted deberá crear una carpeta llamada `T0` en el directorio principal de su carpeta personal y subir su tarea a esa carpeta. En su carpeta `T0` **solo debe incluir el código fuente** necesario para compilar su tarea y un `Makefile`. Se revisará el contenido de dicha carpeta el día Miércoles 2 de septiembre a las 22:00 horas.

Como mínimo, la carpeta deberá incluir:

- El código fuente de `fazwatch`.
- Los archivos `.h` adicionales utilizados por su implementación.
- Un archivo `Makefile`.
- Un archivo `README.md` con cualquier consideración relevante para la corrección.

No deben incluirse en la entrega los ejecutables de las cámaras proporcionadas por el equipo docente, ya que estos serán provistos durante la corrección.

● 11. Formalidades

- La implementación debe realizarse en lenguaje C. Cualquier tarea escrita en otro lenguaje de programación no será revisada.

¹iic2333.ing.puc.cl

- La tarea puede desarrollarse en Linux, macOS o en Windows mediante WSL.
- Su solución debe compilar correctamente utilizando el comando `make` y generar un ejecutable llamado `fazwatch`. Si su programa **no tiene** un `Makefile`, tendrá un descuento de 0.5 décimas en su nota final y corre riesgo que su tarea no sea corregida.
- La solución debe funcionar con las cámaras proporcionadas por el equipo docente y con otras cámaras que respeten el protocolo descrito en este enunciado. No se debe asumir un comportamiento específico de una cámara.
- Deben liberar correctamente la memoria utilizada y cerrar los descriptores creados.
- **NO debe incluir archivos binarios**². En caso contrario, tendrá un descuento de 0.2 décimas en su nota final.
- **NO debe incluir un repositorio de git**. Asegúrate de no incluir la carpeta oculta `.git` dentro de su entrega. En caso contrario, tendrá un descuento de 0.2 décimas en su nota final.
- **NO debe usar VSCode para entrar al servidor**³. En caso contrario, tendrá un descuento de 0,2 puntos en su nota final.
- Si inscribe su pareja de forma incorrecta o no la inscribe, tendrá un descuento de 0.3 décimas en la nota final. Además, si no inscribe se asume que la tarea no se realiza y no será corregido.
- En caso de no cumplir con el formato de entrega, tendrá un descuento de 0.3 décimas en la nota final.

El no respeto de las formalidades o un código extremadamente desordenado podría originar descuentos adicionales, los cuales quedarán a discreción del ayudante corrector. Se recomienda modularizar la solución, utilizar funciones y ocupar nombres de variables descriptivos.

Si la tarea no compila o termina prematuramente debido a errores como *segmentation faults*, obtendrán la nota mínima, pudiendo recorrer modificando líneas de código con un descuento de una décima por cada cuatro líneas modificada, con un máximo de 20 líneas a modificar.

● 12. Evaluación

El puntaje se distribuirá de la siguiente manera:

- **1.2 pts. Lanzamiento de cámaras.** Correcta creación de procesos mediante `fork`, preparación del pipe de heartbeat y ejecución de la cámara correspondiente.
- **1.2 pts. Recepción de heartbeats y funcionamiento no bloqueante.**
- **0.8 pts. Correcto manejo del protocolo de timeout.**
- **0.8 pts. Correcto funcionamiento del comando status.**
- **0.5 pts. Correcto funcionamiento de los comandos pause y resume.**
- **0.5 pts. Correcto funcionamiento de los comandos terminate y shutdown.**
- **0.5 pts. Correcto manejo del ciclo de vida de los procesos. (no dejar proceso zombies)**
- **0.5 pts. Correcto manejo de memoria y file descriptors.**

²Los archivos que resulten del proceso de compilar, como lo son los ejecutables y los *object files*

³Si es que lo hace, puede eliminar la carpeta oculta `.vscode-server` antes de la fecha de entrega.

● Política de atraso

Se puede hacer entrega de la tarea con un máximo de 2 días de atraso. La fórmula a seguir es la siguiente:

$$N_{T_0}^{\text{Atraso}} = \min(N_{T_0}, 7.0 - 0.75 \cdot d)$$

Siendo d la cantidad de días de atraso. Notar que esto equivale a un descuento *soft*, es decir, cambia la nota máxima alcanzable y no se realiza un descuento directo sobre la nota obtenida. El uso de días de atraso no implica días extras para alguna tarea futura, por lo que deben usarse bajo su propio riesgo.

● Preguntas

Cualquier duda, preguntar a través del foro oficial.